

## 第一题

---

求 $\lfloor \sqrt{n} \rfloor$

```
int floorsqrt(int n) {
    if (n < 0) return -1; //负数, 错误情况
    if (n == 0 || n == 1) return n; //0和1的根号即为本身
    //伪二分查找法
    int low = 0, high = n;
    while (high - low > 1) {
        int mid = (low + high) / 2;
        if (mid * mid > n) {
            high = mid;
        }
        else {
            low = mid;
        }
    }
    return low;
}
```

## 第二题

---

1-6: 对于下列各组函数  $f(n)$  和  $g(n)$ , 确定 ( $f(n) = O(g(n))$ ) 或  $f(n) = \Omega(g(n))$  或  $f(n) = \Theta(g(n))$ , 并简述理由。

---

(1)  $f(n) = \log n^2; g(n) = \log n + 5$

$f(n) = \log n^2 = 2 \log n$ 。对任意  $n$  有  $2 \log n \leq 2(\log n + 5)$ ; 对  $n \geq e^5$ , 有  $2 \log n \geq \log n + 5$ , 故  $f(n) = \Theta(g(n))$

(2)  $f(n) = \log n^2; g(n) = \sqrt{n}$

令  $h(n) = f(n) - 2g(n) = 2 \log n - 2\sqrt{n}$ ;  $h'(n) = \frac{2}{n} - \frac{1}{\sqrt{n}} = \frac{2-\sqrt{n}}{n}$ ; 令  $h'(n) > 0$  得  $0 < n < 4$ , 即  $h(n)$  在  $n = 4$  取得极大值;  $h(4) = 4 \log 2 - 4 < 0$ , 故对任意  $n$  有  $h(n) < h(4) < 0$ , 即  $f(n) < 2g(n)$ , 故  $f(n) = O(g(n))$

(3)  $f(n) = n; g(n) = \log^2 n$

对任意  $n$  有  $2n > \log^2 n = 2 \log n$ , 故  $f(n) = \Omega(g(n))$

$$(4) f(n) = n \log n + n; g(n) = \log n$$

$n \log n = \Omega(\log n), n = \Omega(\log n)$ , 故  $n \log n + n = f(n) = \Omega(g(n))$

$$(5) f(n) = 10; g(n) = \log 10$$

两者皆为常数, 故  $f(n) = \Theta(g(n))$

$$(6) f(n) = \log^2 n; g(n) = \log n$$

当  $\log n \geq 1, n \geq \text{某个常数} \epsilon$  时, 有  $f(n) \geq g(n)$ , 故  $f(n) = \Omega(g(n))$

$$(7) f(n) = 2^n; g(n) = 100n^2$$

指数函数增长快于任意多项式, 故  $f(n) = \Omega(g(n))$

$$(8) f(n) = 2^n; g(n) = 3^n$$

对任意  $n > 0$  恒有  $2^n < 3^n$ , 故  $f(n) = O(g(n))$

**1-7: 证明  $n! = o(n^n)$ 。**

---

$$\lim_{n \rightarrow \infty} \frac{n!}{n^n} = \lim_{n \rightarrow \infty} \frac{1*2*...*n}{n*n*...*n} = \lim_{n \rightarrow \infty} \prod_{i=1}^n \frac{i}{n}$$

$i \leq \frac{n}{2}$  时, 有  $\frac{i}{n} \leq \frac{1}{2}$ ;  $\frac{n}{2} < i \leq n$  时, 有  $\frac{i}{n} \leq 1$

故原式  $\leq \lim_{n \rightarrow \infty} (\frac{1}{2})^{\frac{n}{2}} = 0$ ; 又原式  $\geq 0$ , 故由夹逼定理得  $\lim_{n \rightarrow \infty} \frac{n!}{n^n} = 0$

即  $n! = o(n^n)$

## 第三题

---

### 分析以下算法的时间复杂度

---

(1)

```
def func(n):
    i = 1
    while i < n:
        i *= 2
    print(i)
```

每次乘2:  $1, 2, 4, 8, \dots, 2^k < n$

则循环次数:  $k \approx \log_2 n$

循环体内操作是常数时间, 故总体  $T(n) = \Theta(\log n)$

## (2)

```
def T(n):  
    if n <= 1:  
        return 1  
    return 2 * T(n/2) + n
```

(若将代码的 $T(n)$ 视作一个函数而非下文的 $T(n)$ 本身) 递推式:  $T(n) = T(n/2) + \Theta(1)$ ;  $a=1, b=2$ ,

$$n^{\log_b a}$$

$= n^0 = 1$ ; 又  $f(n) = \Theta(1) = \Theta(n^{\log_b a})$ , 故  $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$

## (3)

```
def func(n):  
    sum = 0  
    for i in range(n):  
        for j in range(n):  
            if j % 2 == 0:  
                for k in range(n):  
                    sum += 1
```

最外层  $i$  循环  $n$  次; 每层  $i$  内  $j$  循环  $n$  次, 其中只有当  $j$  为偶数 ( $j \% 2 == 0$ ) 时才进入最内层, 偶数个数  $\approx n/2$ , 故最内层约执行  $n/2$  次; 最内侧  $k$  循环  $n$  次

故总执行次数大约为  $n * \frac{n}{2} * n = \frac{n^3}{2}$  次, 时间复杂度  $T(n) = \Theta(n^3)$

## (4)

```
def func(n):  
    sum = 0  
    for i in range(n):  
        for j in range(n):  
            if j % 2 == 0:  
                for k in range(n):  
                    sum += 1
```

同第三问